Master Thesis Proposal: Cache-Aware Extension of BottleMod for Storage-Hierarchy-Aware Bottleneck Analysis in I/O-Intensive Workflows

Student: Justus Krebs
Supervisor: Joel Witzke
Institution: TU Berlin / Zuse Institute Berlin (ZIB)

March 5, 2026

1 Introduction / Scientific Background / Related Work

1.1 Obstacle

Data-intensive scientific workflows spend a significant fraction of their execution time on I/O operations whose effective throughput depends on the operating system's page cache. When the working set exceeds available memory, cache evictions force data to be re-read from slow block devices, creating performance cliffs that shift bottlenecks from compute to storage. BottleMod [1], a recent progress-based performance model for workflow tasks (ICPE'25), provides fast analytical bottleneck analysis but treats storage bandwidth as a single, constant rate — it cannot represent the performance difference between cached and uncached data access, nor can it predict how task ordering or memory allocation changes effective I/O throughput.

1.2 Motivation

Accurate bottleneck attribution is a prerequisite for informed resource allocation: a scheduler that misidentifies a memory-bandwidth-bound task as disk-bound will waste budget on faster storage instead of allocating more memory. In practice, the page cache introduces a non-linear relationship between available memory and effective I/O bandwidth that existing workflow performance models either ignore entirely or model only at the simulation level (e.g., WRENCH/SimGrid [3, 2]), sacrificing the analytical tractability that makes BottleMod attractive. Bridging this gap — making bottleneck prediction *cache-aware* while preserving BottleMod's lightweight, closed-form character — would enable schedulers and workflow engines to reason about storage hierarchy effects at negligible computational cost.

1.3 Novelty

Existing performance models fall into two camps: (i) bottleneck/roofline models [5] that treat memory as a single bandwidth ceiling without OS cache dynamics, and (ii) page-cache-aware simulation models [6] that capture cache behavior but lack a per-task bottleneck decomposition. Cache-aware DAG scheduling research [7] targets CPU cache hierarchies (L1/L2/L3), not the OS page cache. No existing model integrates page-cache hit-rate dynamics into a progress-based bottleneck predictor. This thesis fills that gap by extending BottleMod with a storage hierarchy layer that translates cache

hit rates into tier-specific bandwidth requirements, enabling the existing progress solver to produce cache-aware predictions and bottleneck timelines.

1.4 Anticipated Impact

A validated cache-aware bottleneck model would allow workflow schedulers and resource managers to: (a) predict how memory allocation affects I/O-bound task runtime, (b) identify when increasing memory (rather than storage bandwidth) removes a bottleneck, and (c) optimize task ordering to maximize page cache reuse. Our preliminary experiments show that cache-friendly task ordering yields a measured 1.09–1.12× speedup on a 4-task ffmpeg workflow — an effect that vanilla BottleMod cannot predict at all.

1.5 Anticipated Contributions

1. A formal extension to the BottleMod model ("BottleMod-CA") that adds logical access profiles, storage tiers, and pluggable cache behavior models.
2. An open-source Python implementation of the extension integrated with the existing BottleMod codebase.
3. Experimental validation on I/O-intensive workloads demonstrating <20% prediction error and correct bottleneck identification.
4. A demonstration that BottleMod-CA enables cache-aware task ordering optimization that vanilla BottleMod cannot express.

1.6 Scientific Background

BottleMod [1] models task execution through requirement functions $R_{D,k}(p)$ (data) and $R_{R,\ell}(p)$ (resources) as functions of progress p, combined with time-dependent input functions $I_{D,k}(t)$ and $I_{R,\ell}(t)$. Progress is computed iteratively: data availability sets an upper bound; resource rates impose speedup constraints. The result is a piecewise-polynomial progress function $P(t)$ and a bottleneck decomposition identifying the limiting resource at each time segment.

Stack distance analysis [4] provides a classic framework for predicting cache hit rates from reuse distance distributions and cache capacity, forming the theoretical basis for our cache behavior models.

The Linux page cache manages file data in DRAM transparently, with LRU-based eviction under memory pressure. Cgroup v2 memory limits allow per-workload control of effective cache capacity, enabling controlled experiments.

1.7 Related Work

Roofline model [5]: bounds performance by compute and memory bandwidth ceilings but does not model OS cache or tiered storage. **WRENCH/SimGrid** [3, 2]: discrete-event workflow simulation with I/O modeling, but no per-task bottleneck decomposition and high computational cost. **Linux page cache modeling** [6]: explicitly models page cache for simulation accuracy but does not integrate with a bottleneck predictor. **Cache-aware DAG scheduling** [7]: targets CPU cache locality in multicore scheduling, not OS page cache in storage hierarchies. **I/O-aware HPC scheduling** [8]: acknowledges storage tiers (burst buffers) in scheduling but does not predict per-task runtimes under varying cache states.

2 Goal of the Thesis / Statement of the Research Problem

2.1 Overarching Goal

Extend the BottleMod performance model with a storage hierarchy layer that enables accurate, cache-aware prediction of task execution time and bottleneck attribution for I/O-intensive workflows under varying memory constraints.

2.2 Research Questions

RQ1: Prediction accuracy. Can a bottleneck-based model predict task runtime when effective read bandwidth is a mixture of memory-speed cache hits and storage-speed cache misses, achieving <20% mean absolute percentage error (MAPE) across varying memory limits and access patterns?

RQ2: Bottleneck identification. Does the cache-aware extension correctly identify the limiting resource (page cache bandwidth vs. disk bandwidth) at each phase of task execution, and can it predict where bottleneck transitions occur as memory pressure changes?

RQ3: Optimization enablement. Can BottleMod-CA predict the runtime impact of cache-aware task ordering (and thereby enable ordering optimizations), where the vanilla model cannot?

2.3 Scope

2.3.1 In Scope

- Single-node storage hierarchies: Linux page cache (DRAM) + block device (SATA HDD/SSD).
- Sequential and random I/O access patterns with varying working set sizes.
- Pluggable cache behavior models: direct hit-rate, stack-distance-based, phase-based.
- Cgroup v2 memory limits as the mechanism to control effective cache capacity.
- Task ordering optimization as an application of the model's predictions.

2.3.2 Out of Scope

- Distributed/networked storage (NFS, object stores, HDFS).
- CPU cache hierarchy effects (L1/L2/L3) — orthogonal and well-covered by existing work.
- Kernel-level page cache modifications or custom eviction policies.
- Multi-node workflow scheduling (focus is on single-node prediction accuracy).

3 Thesis Approach

3.1 Solution Approach

The extension introduces three process-side and environment-side constructs that map onto BottleMod's existing requirement/input function paradigm:

- **Logical Access Profile** $A_k^r(p), A_k^w(p)$: cumulative bytes read/written by dataset k as a function of progress. Process-side descriptor provided by the developer.
- **Storage Tier** with bandwidth input $I_{bw,j}(t)$ and capacity C_j: environment-side descriptor of each tier j (e.g., DRAM at 280 MB/s, SATA at 165 MB/s).
- **Tier Mapping** $H_{k,j}^r(p)$: fraction of dataset k's accesses served by tier j at progress p, computed by a pluggable `CacheBehaviorModel` (direct hit-rate, stack-distance CDF, or phase-based).

The key derivation is: $R'_{(k,j,bw,r)}(p) = H_{k,j}^r(p) \cdot A_k^{r\prime}(p)$, which converts hit fractions and access rates into per-tier bandwidth requirement functions. These are appended to the standard `Task.cpu_funcs` list,

and the tier bandwidth inputs become `in_cpu_funcs` for `TaskExecution`. The existing progress solver and bottleneck decomposition operate unchanged — all cache-awareness is upstream preprocessing.

3.2 Differentiation from State of the Art

Unlike simulation-based approaches (WRENCH, SimGrid), BottleMod-CA produces closed-form piecewise-polynomial predictions with negligible computational cost. Unlike roofline models, it captures dynamic cache state transitions within a single task execution. Unlike CPU-cache-aware schedulers, it targets the OS page cache and file I/O tier, which dominates data-intensive workflow performance.

3.3 Success Criteria

Should we put something here?

References

[1] A. Lößer, J. Witzke, F. Schintke, B. Scheuermann, "BottleMod: Modeling Data Flows and Tasks for Fast Bottleneck Analysis," in *Proc. ICPE*, 2025. https://doi.org/10.1145/3676151.3719382

[2] H. Casanova et al., "Versatile, Scalable, and Accurate Simulation of Distributed Applications and Platforms," *J. Parallel and Distrib. Comput.*, vol. 74, no. 10, 2014.

[3] H. Casanova et al., "Developing Accurate and Scalable Simulators of Production Workflow Management Systems with WRENCH," *Future Generation Computer Systems*, vol. 112, 2020.

[4] R. L. Mattson, J. Gecsei, D. R. Slutz, I. W. Traiger, "Evaluation Techniques for Storage Hierarchies," *IBM Systems Journal*, vol. 9, no. 2, 1970.

[5] S. Williams, A. Waterman, D. Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," *Commun. ACM*, vol. 52, no. 4, 2009.

[6] H. Meurillon, F. Music, G. Pallez, "Modeling the Linux Page Cache for Accurate Simulation of Data-Intensive Applications," in *Proc. IEEE Cluster*, 2021. https://doi.org/10.1109/CLUSTER48925.2021.00058

[7] M. Qamhieh, S. Midonnet, "A Cache-Aware DAG Scheduling Method on Multicores," *J. Systems Architecture*, 2025. https://doi.org/10.1016/j.sysarc.2025.103372

[8] M. Zhao et al., "Scalable I/O-Aware Job Scheduling for Burst-Buffer Enabled HPC Clusters," in *Proc. HPDC*, 2016. https://doi.org/10.1145/2907294.2907316